EXP9: SCILAB ROUTING AND CONGESTION CONTROL

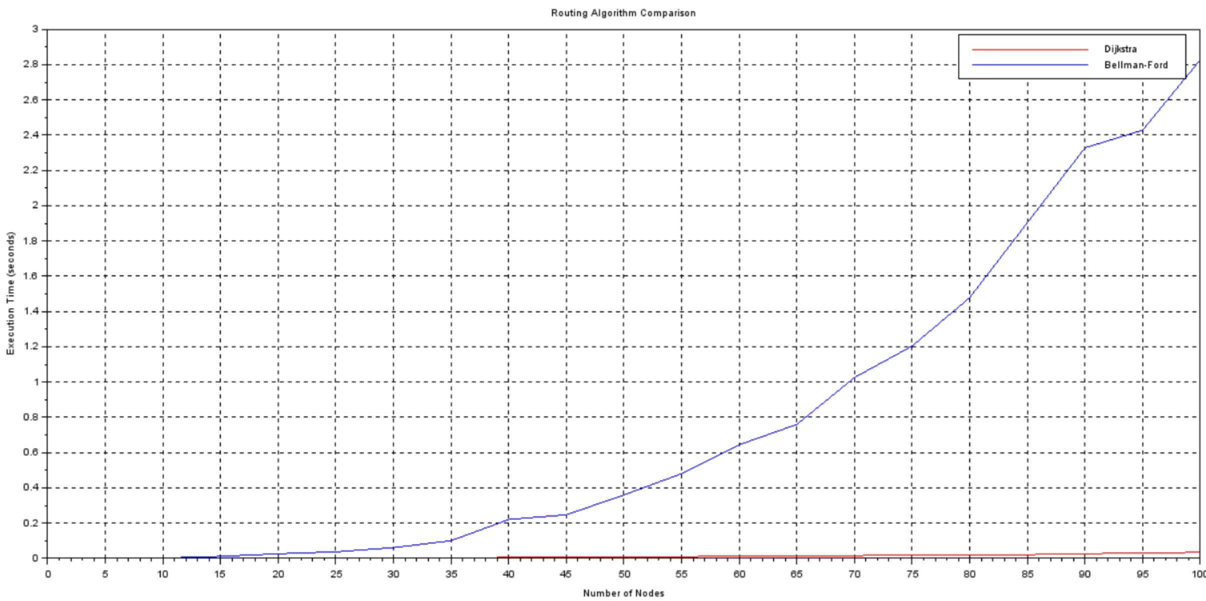
RAVI BALA M.V
Register Number: 24BCE1419

Faculty Name: Subbulakshmi

Lab Slot: L55 + L56

Course Name: Computer Networks Lab
Course Code: BCSE308L

TASK1 :ROUTING ALOGORITHM



CODE:

```
// Adjacency Matrix
function G=adj_mat(n)
    G = grand(n, n, "uin", 1, 10);

    // Make graph sparse
    for i = 1:n
        for j = 1:n
            if rand() > 0.7 then
                G(i,j) = 0;
            end
        end
        G(i,i) = 0;
    end
```

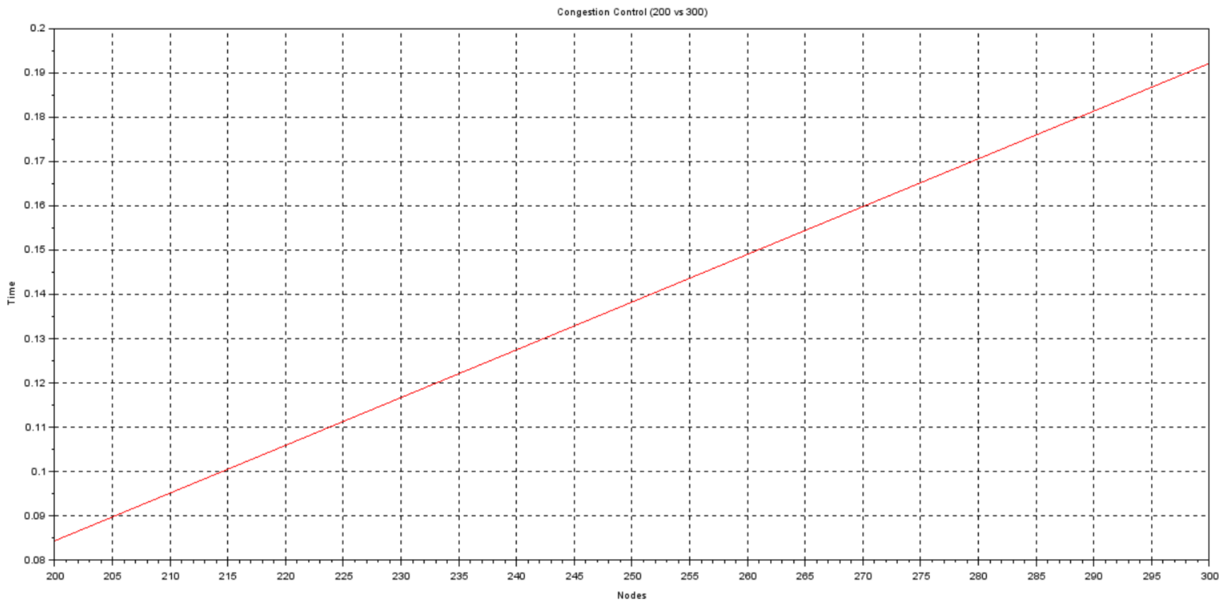
```
endfunction


// Dijkstra Algorithm
function dist=dijkstra(G, src)
    n = size(G,1);
    dist = ones(1,n) * %inf;
    visited = zeros(1,n);
    dist(src) = 0;

    for i = 1:n
        [m,u] = min(dist + visited * max(dist) * 2);
        visited(u) = 1;

        for v = 1:n
            if G(u,v) > 0 & visited(v) == 0 then
                if dist(u) + G(u,v) < dist(v) then
                    dist(v) = dist(u) + G(u,v);
                end
            end
        end
    end
endfunction


// Bellman-Ford Algorithm
function dist=bellman_ford(G, src)
```

```
    n = size(G,1);

    dist = ones(1,n) * %inf;

    dist(src) = 0;


    for k = 1:n-1

        for u = 1:n

            for v = 1:n

                if G(u,v) > 0 then

                    if dist(u) + G(u,v) < dist(v) then

                        dist(v) = dist(u) + G(u,v);

                    end

                end

            end

        end

    end

endfunction


nodes = 5:5:100;

time_dijkstra = [];

time_bellman = [];


for n = nodes


    G = adj_mat(n);
```

```
    // -------- Dijkstra Timing --------

    tic();

    dijkstra(G, 1);

    time_dijkstra($+1) = toc();


    // -------- Bellman-Ford Timing --------

    tic();

    bellman_ford(G, 1);

    time_bellman($+1) = toc();


end


clf();


// Plot Dijkstra (RED)

plot(nodes, time_dijkstra, '-r');


// Plot Bellman-Ford (BLUE)

plot(nodes, time_bellman, '-b');


xlabel("Number of Nodes");

ylabel("Execution Time (seconds)");

title("Routing Algorithm Comparison");

legend("Dijkstra", "Bellman-Ford");

xgrid();
```

TASK 2:

G200 v G300 node;



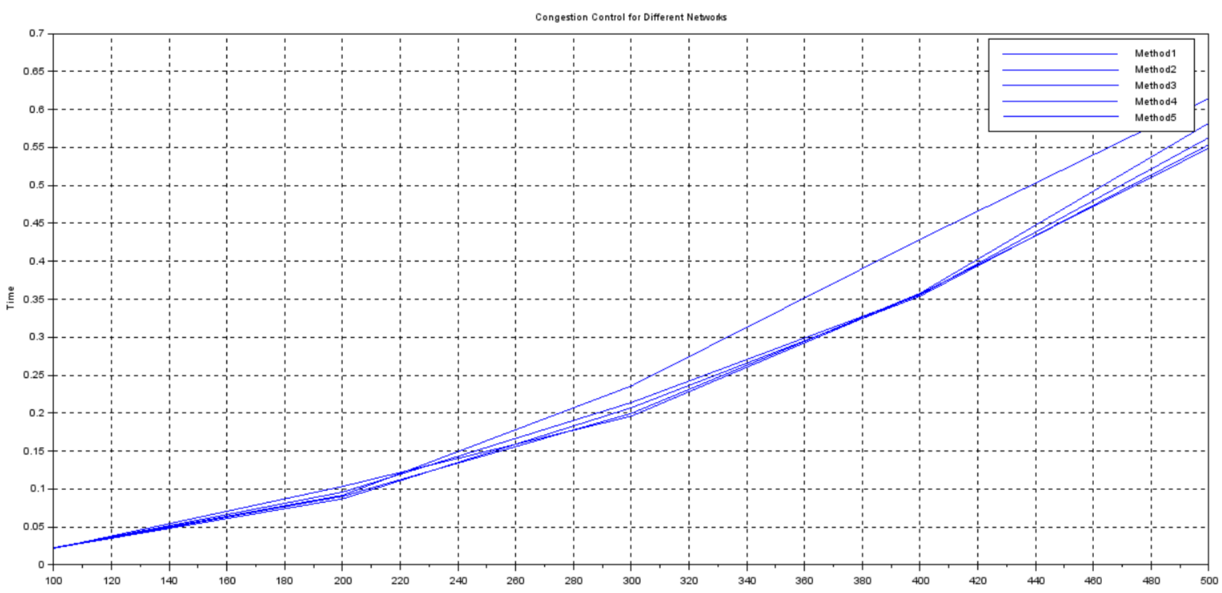
CODE:

```
function G=adj_mat(n)
    G = grand(n, n, "uin", 1, 10);

    for i = 1:n
        for j = 1:n
            if rand() > 0.7 then
                G(i,j) = 0;
            end
        end
        G(i,i) = 0;
    end
```

```
endfunction


function load=congestion_control(G)
    n = size(G,1);
    load = zeros(1,n);

    for i = 1:n
        for j = 1:n
            if G(i,j) > 0 then
                load(i) = load(i) + G(i,j);
            end
        end
    end
endfunction


G200 = adj_mat(200);
G300 = adj_mat(300);


// Timing
tic();
congestion_control(G200);
time200 = toc();


tic();
congestion_control(G300);
```

```
time300 = toc();


// Plot

nodes1 = [200 300];

time1 = [time200 time300];


clf();

plot(nodes1, time1, '-r');

xlabel("Nodes");

ylabel("Time");

title("Congestion Control (200 vs 300)");

xgrid();
```

500 NODES +REDUCTION;



CODE:

```
function G=adj_mat(n)
    G = grand(n, n, "uin", 1, 10);

    for i = 1:n
        for j = 1:n
            if rand() > 0.7 then
                G(i,j) = 0;
            end
        end
        G(i,i) = 0;
    end
endfunction

function load=congestion_control(G)
    n = size(G,1);
    load = zeros(1,n);

    for i = 1:n
        for j = 1:n
            if G(i,j) > 0 then
                load(i) = load(i) + G(i,j);
            end
        end
    end
```

```
endfunction

G200 = adj_mat(200);
G300 = adj_mat(300);

// Timing
tic();
congestion_control(G200);
time200 = toc();

tic();
congestion_control(G300);
time300 = toc();

// Plot
nodes1 = [200 300];
time1 = [time200 time300];

clf();
plot(nodes1, time1, '-r');
xlabel("Nodes");
ylabel("Time");
title("Congestion Control (200 vs 300)");
xgrid();
```

```
// Create 5 different networks
G1 = adj_mat(500);
G2 = adj_mat(500);
G3 = adj_mat(500);
G4 = adj_mat(500);
G5 = adj_mat(500);

sizes = [500 400 300 200 100];
time_all = zeros(5, length(sizes));

// Loop through methods
for m = 1:5

    if m == 1 then G = G1;
    elseif m == 2 then G = G2;
    elseif m == 3 then G = G3;
    elseif m == 4 then G = G4;
    else G = G5;
    end

    // Reduce size
    for i = 1:length(sizes)
        n = sizes(i);
        G_sub = G(1:n,1:n);
```

```
        tic();
        congestion_control(G_sub);
        time_all(m,i) = toc();
    end
end

// Plot all methods
clf();
for m = 1:5
    plot(sizes, time_all(m,:), '-');
end

xlabel("Nodes");
ylabel("Time");
title("Congestion Control for Different Networks");
legend("Method1","Method2","Method3","Method4","Method5");
xgrid();
```